



Instituto Infnet

# Disciplina - Inteligência Artificial e Machine Learning

*Etapa 9 - Utilizando recursos de dispositivos móveis, autenticação e estilos*

**Professor:** Tiago Cariolano de Souza Xavier

**Email** : [tiago.xavier@prof.infnet.edu.br](mailto:tiago.xavier@prof.infnet.edu.br)

# ImagePicker

- **ImagePicker (expo-image-picker):** biblioteca react native que permite:
  - Selecionar imagens da galeria (camera roll)
  - Tirar fotos usando a câmera do dispositivo
  - Retornar a imagem selecionada/capturada como uma URI
- **Vantagens:**
  - Simplicidade de uso
  - Integração com Expo (sem configuração nativa)
  - Suporte a Android, iOS e Web
  - Evita lidar diretamente com APIs nativas



# ImagePicker: importação e permissões

- **Importação:** A biblioteca é importada como um objeto com várias funções assíncronas
  - `import * as ImagePicker from 'expo-image-picker';`
- **Permissões:** Antes de acessar câmera ou galeria, é necessário solicitar permissão
  - **Galeria:**
    - `await ImagePicker.requestMediaLibraryPermissionsAsync();`
  - **Câmera:**
    - `await ImagePicker.requestCameraPermissionsAsync();`



# ImagePicker: Utilização

- **Selecionar imagem da galeria:**

- `const result = await ImagePicker.launchImageLibraryAsync();`
- **Configuração opcional:**
  - ```
{  
    mediaTypes: ImagePicker.MediaTypeOptions.Images,  
    allowsEditing: true,  
    quality: 1  
}
```

- **Tirar foto com a câmera:**

- `const result = await ImagePicker.launchCameraAsync();`

- **Fluxo:**

- *1.Solicita permissão. 2.Abre câmera do sistema. 3.Usuário tira foto. 4.Retorna imagem*

# ImagePicker: Exibição de imagem e exemplo

- **Exibir imagem:**

- `<Image source={{ uri: imageUrl }} />`

- **Exemplo:**

- ```
const [imageUrl, setImageUri] = useState(null);  
async function escolherImagem() {  
  const result = await ImagePicker.launchImageLibraryAsync();  
  if (!result.canceled) {  
    setImageUri(result.assets[0].uri);  
  }  
}
```

- **Retorno:**

- **canceled:** indica se o usuário cancelou
- **assets:** lista com os arquivos selecionados
- **uri:** caminho da imagem no dispositivo

- **Exibir Imagem:** `<Image source={{ uri: imageUrl }} />`

# ImagePicker: Exemplo Galeria e Câmera

- GaleriaCamera.js:

```
import React, { useState } from 'react';  
import { Button, Image, StyleSheet, Alert, View } from 'react-native';  
import * as ImagePicker from 'expo-image-picker';  
  
export default function GaleriaCamera() {  
  const [imageUri, setImageUri] = useState(null);
```



# ImagePicker: Exemplo Galeria e Câmera

- GaleriaCamera.js (continuação):

```
// GALERIA
```

```
async function escolherImagem() {  
  const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync()  
  
  if (status !== 'granted') {  
    Alert.alert('Permissão negada');  
    return;  
  }  
  
  const result = await ImagePicker.launchImageLibraryAsync();  
  
  if (!result.canceled) {  
    setImageUri(result.assets[0].uri);  
  }  
}
```

# ImagePicker: Exemplo Galeria e Câmera

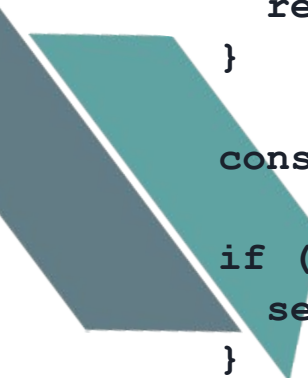
- GaleriaCamera.js (continuação):

```
// CÂMERA
async function tirarFoto() {
  const { status } = await ImagePicker.requestCameraPermissionsAsync();

  if (status !== 'granted') {
    Alert.alert('Permissão negada');
    return;
  }

  const result = await ImagePicker.launchCameraAsync();

  if (!result.canceled) {
    setImageUri(result.assets[0].uri);
  }
}
```

A decorative graphic in the bottom-left corner consisting of two overlapping parallelograms. The front one is a teal color, and the back one is a darker blue-grey color. They are oriented diagonally, pointing towards the bottom-right.



# ImagePicker: Exemplo Galeria e Câmera

- GaleriaCamera.js (continuação):

```
return (  
  <View style={styles.container}>  
    <View style={styles.botao}>  
      <Button title="Galeria" onPress={escolherImagem} />  
    </View>  
  
    <View style={styles.botao}>  
      <Button title="Câmera" onPress={tirarFoto} />  
    </View>  
  
    {imageUri && (  
      <Image source={{ uri: imageUri }} style={styles.image} />  
    )}  
  </View>  
);  
}
```

# ImagePicker: Exemplo Galeria e Câmera

- GaleriaCamera.js (continuação):

```
const styles = StyleSheet.create({
  container: {
    width: "50%",
    flexDirection: 'column',
  },

  botao: {
    width: '100%', // cada botão ocupa 25%
  },

  image: {
    width: '100%', // ocupa metade da tela
    height: 200,
    marginTop: 10,
  },
});
```

# Expo-Camera

- **Expo-Camera:** biblioteca React que permite:
  - Acessar a câmera do dispositivo
  - Exibir preview em tempo real
  - Capturar fotos diretamente no app
  - Funcionar em Android, iOS e Web
- **Quando usar:**
  - Preview da câmera na tela
  - Controle de captura (foto em tempo real)
  - Experiência mais próxima de apps nativos
  - Webcam no navegador
- **Instalação:**
  - `npm install expo-camera`
  - `npx expo install expo-camera`

# Expo-Camera: importação e permissões

- **Importação:**

- `import { CameraView, useCameraPermissions } from 'expo-camera';`

- **CameraView** → componente visual da câmera

- **useCameraPermissions** → hook para permissões

- **Permissões (obrigatório):**

- `const [permission, requestPermission] = useCameraPermissions();`

- **Verificação:**

- `if (!permission.granted) {  
    requestPermission();  
}`



# Expo-Camera: uso

- **Utilização:**

- `<CameraView style={{ flex: 1 }} />`
  - Preview da câmera em tempo real
  - Webcam no navegador
  - Câmera frontal ou traseira no celular

- **Captura de imagem:**

- `const photo = await cameraRef.current.takePictureAsync();`
  - `photo.uri` → caminho da imagem capturada



# Exemplo captura de câmera

## navegador

- Camera.js:

```
import React, { useRef, useState } from 'react';
import { View, Button, Image, StyleSheet, Alert } from 'react-native';
import { CameraView, useCameraPermissions } from 'expo-camera';

export default function Camera() {
  const [permission, requestPermission] = useCameraPermissions();
  const [photoUri, setPhotoUri] = useState(null);
  const cameraRef = useRef(null);
```

A decorative graphic in the bottom-left corner consisting of two overlapping triangles. The front triangle is teal and points downwards and to the right. The back triangle is a darker teal/grey and points downwards and to the left.

# Exemplo captura de câmera navegador

- Camera.js (continuação):

```
async function tirarFoto() {  
    if (!cameraRef.current) return;  
  
    try {  
        const photo = await cameraRef.current.takePictureAsync();  
        setPhotoUri(photo.uri);  
    } catch {  
        Alert.alert('Erro', 'Não foi possível tirar a foto.');    }  
}
```

A decorative graphic in the bottom-left corner consisting of two overlapping parallelograms. The front one is a teal color, and the back one is a darker grey-blue color. They are oriented diagonally.

# Exemplo captura de câmera navegador

- Camera.js (continuação):

```
if (!permission) {  
  return <View/>;  
}
```

```
if (!permission.granted) {  
  return (  
    <View style={styles.container}>  
      <Button title="Permitir câmera" onPress={requestPermission} />  
    </View>  
  );  
}
```

A decorative graphic in the bottom-left corner consisting of overlapping teal and grey parallelograms.



# Exemplo captura de câmera navegador

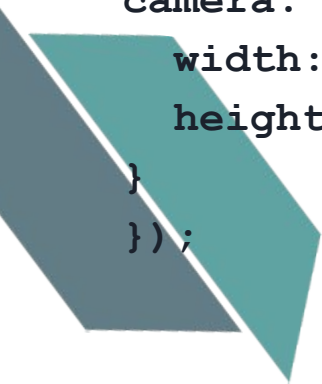
- Camera.js (continuação):

```
return (  
  <View style={styles.container}>  
    {!photoUri ? (  
      <>  
        <Button title="Tirar foto" onPress={tirarFoto} />  
        <CameraView ref={cameraRef} facing="front" style={styles.camera}/>  
  
      </>  
    ) : (  
      <>  
        <Button title="Tirar outra foto" onPress={() => setPhotoUri(null)} />  
        <Image source={{ uri: photoUri }} style={styles.camera}/>  
  
      </>  
    )}  
  </View>  
) ;  
}
```

# Exemplo captura de câmera navegador

- Camera.js (continuação):

```
const styles = StyleSheet.create({
  container: {
    width: '50%',
    height: 300,
  },
  camera: {
    width: "100%",
    height: 300
  }
});
```

Decorative geometric shapes consisting of overlapping teal and dark blue triangles in the bottom-left corner.

# Diferenças de estilo iOS e Android

- **Por que é necessário diferenciar?**
  - iOS e Android possuem guidelines diferentes de UI
  - Componentes nativos têm comportamentos distintos
  - **Objetivo:**
    - Manter consistência com o sistema
    - Melhorar UX (experiência do usuário)
- **Como o React Native resolve o problema?**
  - API **Platform**: Permite identificar o sistema em runtime
    - **Platform.OS**
      - "ios"
      - "android"
      - "web"



# Uso (Platform.OS)

- **Condicional simples:**

- Diferença pequena
- Lógica simples
- **Exemplo:**

- ```
import { Platform } from 'react-native';  
const color = Platform.OS === 'ios' ? 'blue' : 'green';
```



# Uso (Platform.OS)

- **Abordagem para estilos:**

`Platform.select()`

- **Vantagens:**

- Código mais limpo
- Melhor organização
- Escalável

- **Exemplo:**

- ```
...Platform.select({  
  ios: { backgroundColor:  
    'blue' },  
  android: {  
    backgroundColor: 'purple' }  
});
```

- **Exemplo completo:**

```
const styles = StyleSheet.create({  
  button: {  
    padding: 15,  
    ...Platform.select({  
      ios: {  
        backgroundColor: 'blue',  
      },  
      android: {  
        backgroundColor: 'purple',  
      },  
    })  
  },  
});
```

# Uso (Platform.OS)

- **Arquivos por plataforma:** `.ios.js` / `.android.js` / `web.js`
  - Layout diferente
  - Comportamento diferente
- **Dar ao mesmo componente nomes diferentes com sufixo da plataforma**
  - `PostItem.ios.js`
  - `PostItem.android.js`
- **Importação:**
  - `import PostItem from './PostItem';`
    - Sem colocar `.ios.js` nem `.android.js` no import
    - React Native faz a resolução automática



# Definição da Ordem de Apresentação

- 1. Lista de alunos será ordenada aleatoriamente (em aula)
- 2. Ordem de apresentação do grupo é definida pelo aluno do grupo mais bem colocado
  - Exemplo:
    - Grupo 1: B, A, F
    - Grupo 2: E, C, D
    - Ordem aleatória: D, E, F, B, C, A
    - Ordem grupos: Grupo2, Grupo 1
- Grupos serão chamados para apresentar um a um
- Se o grupo não estiver presente no primeiro dia e for chamado, perde a apresentação
- Se algum grupo quiser apresentar primeiro, pode fazê-lo desde que os outros grupos estejam de acordo
- Todos os grupos devem estar presentes nos dois dias, exceto já tenham apresentado

# Como é a apresentação?

- Cada integrante de um grupo irá apresentar uma parte do trabalho (textual ou implementação)
- Grupo deve preparar uma apresentação de slides para exibir
  - Apresentação deve descrever o que foi feito:
    - Na parte textual (Seções do texto, backlog, histórias de usuário, etc)
    - Na parte de implementação (componentes, navegação, screens, etc)
- Cada integrante do grupo deve compartilhar sua tela, de cada vez, e apresentar sua parte dos slides
- Após apresentação dos slides, um integrante do grupo deve compartilhar sua tela e executar o sistema e mostrar o seu funcionamento
- Professor irá fazer perguntas:
  - Relacionadas aos feedbacks dos TPs
  - Relacionadas ao conteúdo textual
  - Relacionadas à implementação
- Código-fonte e pdf devem estar prontos para serem compartilhados, a fim de tirar alguma dúvida do professor



# Como é a avaliação?

- **Uma parte da avaliação é relacionada ao conteúdo do trabalho**
  - **Parte textual (PDF)**
  - **Parte de implementação (código-fonte) e sistema funcionando**
- **Uma parte da avaliação é relacionada à apresentação**
  - **Explicação com slides**
  - **Funcionamento do Sistema**
  - **Dúvidas do professor**

